OBLIG-3

02 November 2026 18:15

---

1. Oppgave 1.
   a. Høyeste verdi: 2
      i. Skjer dersom tråd A fullfører hele økningen før tråd B leser verdien:
      ii. A leser '0', øker til '1' og skriver '1' tilbake.
      iii. B leser '1', øker til '2' og skriver '2' tilbake.
   b. Laveste verdi: 1
      i. Skjer dersom begge trådene leser 0 før noen skriver tilbake. Begge beregner da '0 + 1' og skriver '1'.
      ii. Den ene økningen går tapt, kalles for tapto oppdatering og skyldes en race condition.
   c. Rekkefølge som gir laveste verdi.
      i. Hver tråd har sin egen registertilstand. %eax i tråd 'A' og tråd 'B' kan derfor inneholde forskjellige verdier, mens teller er felles.
      ii.

Steg	Tråd	Instruksjon	Resultat	Felles Teller
1	A	movl teller, %eax	'A' leser '0'	0
2	B	movl teller, %eax	'B' leser '0'	0
3	A	addl $1, %eax	'A' sitt register blir '1'	0
4	B	addl $1, %eax	'B' sitt register blir '1'	0
5	A	movl %eax, teller	'A' skriver '1'	1
6	B	movl %eax, teller	'B' skriver også '1'	1

   d. Med instrukksjonen oppgaven oppgir:
      i. 'addl $1, teller'
      ii. Og 'taskset -c 0' kjører begge trådene på samme logiske CPU.
      iii. Bare en av dem kjører om gangen og et trådbytte skjer ikke midt i instruksjonens utførelse.
      iv. Den første økningen fullføres før den andre utføres og sluttverdien blir 2.
      v. ---
      vi. Med 'taskset -c 0,1' kan trådene kjøre samtidig på hver sin logiske CPU.
      vii. Minneoppdateringen består av å lese, endre og skrive.
      viii. Uten ekstra synkronisering er hele operasjonen ikke atomisk ovenfor andre kjerner.
      ix. Begge kan derfor lese 0 og ende med å skrive 1.
   e. Hele økningen må beskyttes med en atomisk operasjon eller en mutex.
      i. Kan utrykkes som: (på x86)
      ii. 'lock addl $1, teller'
      iii. Da kan ikke to kjerner utføre motstridende oppdateringer av teller samtidig på en måte som mister en økning.

---

2. Oppgave 2
   a. En rekkefølge som fører til en deadlock:
      i.

Steg	Tråd 1 'A -> B'	Tråd 2 'B -> A'
1	Låser konto A	
2		Låser konto B
3	Prøver å låse B, må vente	
4		Prøver å låse A, må vente

      ii. Tråd 1 holder låsen på 'A' og venter på 'B', mens tråd 2 holder låsen på 'B' og venter på 'A'.
      iii. Ingenm slipper låsen og begge står fast.
   b. Her oppstår sirkulær venting, Tråd 1 venter på en ressurs som tråd 2 holder, og vice versa.
      i. Alle 4 betingelsene for deadlock er oppfylt.
         1) Gjensidig utelukkelse: Bare en tråd kan holde en bestemt kontolås om gangen.
         2) Hold og vent: Trådene holder en lås mens de venter på en annen.
         3) Ingen tvungen frigivelse: Låsene kan ikke tas fra trådene; de må selv slippe dem.
         4) Sirkulær venting: Trådene venter på hveranmdre i en sirkel.
   c. Regler som hindrer deadlock:
      i. Gi alle kontoene et unikt nummer, og bestem at alle tråder alltid låser kontoen med lavest nummer først, uansett hvilken vei pengene overføres.
      ii. 'A = 1' 'B = 2' 'C = 3'
      iii.

Tråd	Overføring	Rekkefølge på lås
1	'A' -> 'B'	'A' -> 'B'
2	'B' -> 'A'	'A' -> 'B'
3	'B' -> 'C' / 'C' -> 'B'	'B' -> 'C'

      iv. Hvis tråd 1 holder 'A', må tråd 2 vente før den får tatt noen av låsene. Dermed kan ikke tråd 2 holde 'B' og samtidig vente på 'A'.
      v. Løsningen virker også med konto 'C' så lenge alle trådene følger samme nummerering. Ventingen går alltid fra lavere til høyere nummer og kan derfor ikke danne en sirkel.
   d. Nummerering hindrer deadlock gjennom en fast regel. Tilfeldig ventetid gir ingen garanti for fremdrift:
      i. Trådene kan kollidere flere ganger uten å fullføre.
      ii. Dette kan føre til en "Livelock", og enkelte tråder kan bli forbigått flere ganger. (Starved)
      iii. Hvist den skal hjelpe må tråden dessuten slippe den første låsen dersom den ikke får den andre.
      iv. Å bare vente en tilfeldig stund mens den beholder låsen, løser ikke deadlocken.

---

3. Oppgave 3:
   a. Oppgave 1:
      i. Make main-race
      ii. Valgrind --tool=helgrind ./main-race
         1) Tråd 1 er hovedtråden, den kjører 'main()'
         2) Tråd 2 er arbeidstråden, den kjører 'worker()'
         3)

Tråd	Sted	Operasjon
1	Main-race.c:15	Balance++
2	Main-race.c:8	Balance++

         4) Første feilmld: "Possible data race during read of size 4..."
         5) Hovedtråden leser balance på linje 15. Denne tilgangen er i konflikt med arbeidtrådens skriving på linje 8.
         6) Andre feilmld: "Possible data race during read of size 4..."
         7) Hovedtrådens skriving i linje 15 er også i konflikt med arbeidtrådens skriving på linje 8.
         8) Får både lese og skriveadvarsler fordi balance++ innebærer:
            a) Lese verdien
            b) Øke verdi med en
            c) Skrive verdi tilbake
         9) ERROR SUMMARY: 2 errors from 2 contexts
            a) Samme årsåk: usynkronisert tilgang til balance.

---

3. Oppgave 3:
   a. Oppgave 2:
   b. Forsøk 1 (fjern en av de to økningene.)
      i. Kommenterer ut 'balance++' inne i 'main()':
      ii. Helgrind rapporterer 0 feil.
      iii. Økningen i 'main()' ble fjernet, bare 'worker()' som endrer 'balance'.
   c. Forsøk 2 (lås rundt bare en av økningene):
      i. Reaktiverer 'balance++' i 'main()'
      ii. Legger til en global mutex rett under int balance = 0;
      iii. pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
      iv. Locker worker
      v. 2 datarace varsler, 'worker()' holder låsen, 'main()' gjør ikke det.
      vi. Låsen hindrer kun andre tråder som prøver å ta samme låsen, derfor kan hovedtråden endre balance uten å vente.
   d. Forsøk 3 (Lås rundt begge økningne)
      i. Helgrind finner ingen feil. Begge trådene bruker samme mutex.
      ii. Dette leder til at bare en om gangen kan utføre balance.

3. Oppgave 3:
   a. Oppgave 3:
      i. Undersøker main-deadlock.c
      ii. Låsene er i motsatt rekkefølge
      iii.

Tråd	Låser
1	M1
2	M2
Tråd	Venter på
1	M2 (holdes av tråd 2)
2	M1(holdes av tråd 1)

      iv. Begge venter på hverandre og kommer aldri frem til 'unlock()'
      v. Trenger ikke å skje hvergang, er avhengig av rekkefølgen trådene kjører I

3. Oppgave 3:
   a. Oppgave 4
      i. Helgrind fant feil låserekkefølge som kan føre til deadlock. Melding
      ii. 'Thread #3: lock order "0x4004040 before 0x4004080" violated'
      iii. Identifiseres slik:
      iv.

Adresse	Lås
0x4004040	M1
0x4004080	M2

      v. Meldingen betyr derfor at tråd 3 tok 'm2' før 'm1', mens Helgrind tidligere hadde observert rekkefølgen 'm1' før 'm2'.

---

4. Oggpave 4:
   a. Forkcount.c
      i. 'Fork()' opretter barnet, og barnet får sitt eger adresserom med en kopi av g_ant.
      ii. Foreldrenes endringer påvirker derfor ikke barnets teller. Dette ser jeg når forelderen når 40, mens barnet bare har nådd 19, og fortsetter videre på egenhånd.
      iii. Linjene kommer blandet men det skyldes at OS veksler mellom prosessene.
   b. Threadcount.c
      i. Trådene deler samme g_ant. Begge bidrar derfor til en felles telling mot 30.
      ii. Bare 2nd thread skriver ut noe, og tallet 2 mangler, det betyr ikke at første tråden aldri kjørte.
      iii. Forklaringer:
         1) Andre tråd øker telleren til 1 og skriver ut.
         2) Første tråd øker telleren til 2, men avbrytes før utskrift.
         3) Andre tråd fortsetter med 3,4…30.
         4) Den kaller 'exit(0)', som avslutter før tråd 1 får skrevet ut 2.

---

5. Oppgave 5:
   a. X må være 0.
      i. Sem_init(&s, 0, 0);
      ii. Da har semaforen ingen tilgjeneglige tillatelser. 'parent' må vente i 'sem_wait()' til 'childæ har kalt 'sem, _post()'.
      iii. Utskriften blir alltid:
      iv.

Parent: begin	
Child	
Parent: end	

      v. Parent: begin skrives før barnet oprettes, og child skrives før barnet signaliserer med sem_post().
   b. De to mulige rekkefølgene med x = 0
      i. Tilfelle A: Barnet rekker å signalisere før parent kaller 'sem_wait()'.
      ii.

Steg	Hendelse	Semaforverdi
1	Semafor initialiseres	0
2	Parent skriver 'parent: begin' og opretter barnet	0
3	Child skriver 'child'	0
4	Child kaller 'sem_post()'	1
5	Parent kaller 'sem_wait()' og tar tillatelsen Uten å blokkere	0
6	Parent skriver 'parent: end'	0

      iii. Semaforen husker signalet, selv om parent ikke har begynt å vente enda.
      iv. Tilfelle B_ Parent kommer til 'sem_wait()' først.
      v.

Steg	Hendelse	Semaforverdi
1	Semafor initialiseres	0
2	Parent skriver 'parent: begin' og opretter barnet	0
3	Parent kaller 'sem_wait()' og blokkeres	0 tilgjengelige tillatelser
4	Child skriver 'child'	0 tilgjengelige tillatelser
5	Child kaller 'sem_post()', slik at parent kan vekkes Og fullføre 'sem_wait()'	0 etter at parent har tatt tillatelsen
6	Parent skriver 'parent: end'	0

   c. Hva skjer med x = 1?
      i. Da finnes det allerede en tillatelse, parent kan derfor passere 'sem_wait()' uten signal fra child.
      ii. Da kan 'parent: end' komme før 'child' Parent kan også avslutte hele prosessen ved 'return 0' før barnet rekker å skrive noe.
   d. Hvorfor ikke brute mutex?
      i. En mutex brukes for å beskytte en kritisk sekjson og har eierskap: altså tråden som låser mutexen må selv låse den opp.
      ii. I dette tilfellet trenges det at child gir parent tillatelse til å fortsette.
      iii. En semafor tillater at en tråd kaller 'sem_wait()' og en annen kaller sem_post()'.

---

6. Oppgave 6:
   a. Hvorfor virker ikke orginalen?
      i. 'pthread_create()' starter en arbeidstråd, og hovedtråden kaller også 'writeloop(NULL)'. Begge kjører dermed funksjonen og deler g_ant.

ii. Mulig forløp:
    1) Tråd A øker 'g_ant' fra 0 til 1.
    2) A sover i 'usleep()'.
    3) Tråd B øker g_ant til 2.
    4) A våkner og skriver ut den nåværene verdien 2.
iii. Dermed kan tall bli hoppet over eller skrevet flere ganger. Begga kan også passere grensesjekken når telleren er '9'. Slik at den kan økes for langt.
iv. I tillegg avslutter exit(0) hele prosessen, også den andre tråden. Det kan stoppe utskiften før den er ferdig.

b. Hvordan løser nye koden problemet?
i. Grensesjekken er innenfor låsen: Ingen kan sjekke en gammel verdi som kunne ha ledet til å øke telleren forbi 10.
ii. Økning og utskrift er innenfor samme lås: Ingen annen tråd kan endre telleren mellom disse operasjonene.
iii. Låsen slippes også før 'break': Den andre tråden kan komme inn og oppdage at tellingen er ferdig.
iv. 'return NULL' avslutter funk: Arbeidstråden avsluttes uten å stoppe hele prosessen. Hovedtråden kan nå 'pthread_join()'.
v.

Kompilasjon:	gcc -Wall -g -pthread writeloop_korrekt.c -o writeloop_korrekt
Helgrind:	valgrind --tool=helgrind ./writeloop_korrekt

---

7. Oppgave 7:
a. Kjører orginalen med to readers og en writer:
i. Readers leser 'value', writer øker den.
ii. Får 20 lesninger og 10 skrivinger.
iii. Flere readers kan lese samtidig, en writer trenger eksklusiv tilgang.
b. Kjører med ti readers og to writers:
i. Originalen slipper inn nye readers selv om en writer venter.
ii. Så lenge en reader er aktiv er 'writelock' opptatt.
iii. Writer starvation oppstår da det finnes en stadig størm av readers.
iv. Alle 100 lesningnene skjer før første writer får skrive.
c. Ny forbedret kode:
i. Begge writers skriver 10 hanger hver.
ii. Skriveveridene øker fra 1 til 20 uten manglede oppdateringer.
iii. Prog fullfører med 'end: value 20' som forventet.
iv. Readers endrer ikke value.
v. Kjørte helgrind, null feil.

8. Forandringer:
a. La til en <port> kalt 'turnstile': Porten gjør at en writer kan stoppe nye readers mens den venter på at eksisterende readers skal bli ferdige.
b. Programmet tillater:
i. Flere readers som leser samtidig.
ii. Bare en writer om gangen.
iii. Ingen readers samtidig med en writer.
iv. Nye linjer = 6
v.

Sted	Ny linje	Funksjon
I struktur	Sem_t turnstile;	Opretter en semafor, brukes som port for readers og writers.
I rwlock_init()	Sem_init(&lock->turnstile,1);	Init porten som åpenn, med en tilgjengelig tillatelse.
Før readeren registrerer seg	Sem_wait(&lock->turnstile);	Tar porten, eller venter dersom en writer holder den stengt.
Rett etterpå	Sem_post(&lock->turnstile);	Slipper porten når readeren har passert, neste tråd kan passere.
Før writer tar writelock	Sem_wait(&lock->turnstile);	Stenger porten for nye readers mens writer venter på skrivetilgang og utfører skrivingen.
Når writer avslutter skriving	Sem_post(&lock-turnstile);	Åpner port igjen, neste ventede reader eller writer kan passere.